```
import marimo

__generated_with = "0.25.2"
app = marimo.App()


@app.cell
def _():
    import marimo as mo

    file = mo.ui.file(filetypes=[".csv"])
    file
    return file, mo


@app.cell
def _(file):
    import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt
    import io

    df = pd.read_csv(io.BytesIO(file.contents()))
    df.shape
    return df, np, pd, plt


@app.cell
def _(df):
    df.head()
    return


@app.cell
def _(df):
    df["readmitted"].value_counts()
    return


@app.cell
def _(df):
    df["readmitted_30"] = (df["readmitted"] == "<30").astype(int)

    df["readmitted_30"].value_counts()
    return


@app.cell
def _(df):
    df["discharge_disposition_id"].value_counts().sort_index()
```

```
    return


@app.cell
def _(df):
    discharge_codes = [11, 13, 14, 19, 20, 21]

    df["discharge_disposition_id"].value_counts().reindex(
        discharge_codes, fill_value=0
    )
    return


@app.cell
def _(df):
    excluded_codes = [11, 13, 14, 19, 20, 21]

    df_eligible = df[
        ~df["discharge_disposition_id"].isin(excluded_codes)
    ].copy()

    print("Original encounters:", len(df))
    print("Excluded encounters:", len(df) - len(df_eligible))
    print("Eligible encounters:", len(df_eligible))
    print("Unique patients:", df_eligible["patient_nbr"].nunique())
    return (df_eligible,)


@app.cell
def _(df_eligible):
    print("Readmission counts:")
    print(df_eligible["readmitted_30"].value_counts())

    print("\n30-day readmission rate:")
    print(round(df_eligible["readmitted_30"].mean() * 100, 2), "%")
    return


@app.cell
def _(df_eligible, np):
    missing_counts = (
        df_eligible.replace("?", np.nan)
        .isna()
        .sum()
        .sort_values(ascending=False)
    )

    missing_counts[missing_counts > 0]
    return
```

```
@app.cell
def _(df_eligible):
    print("Exact duplicate rows:", df_eligible.duplicated().sum())

    print(
        "Duplicate encounter IDs:",
        df_eligible["encounter_id"].duplicated().sum()


    )

    print(
        "Patients with multiple encounters:",
        (df_eligible["patient_nbr"].value_counts() > 1).sum()
    )
    return


@app.cell
def _(df_eligible):
    print("Available columns:")
    for column in df_eligible.columns:
        print(column)
    return


@app.cell
def _(df_eligible):
    predictors = [
        "age",
        "gender",
        "race",
        "admission_type_id",
        "time_in_hospital",
        "num_lab_procedures",
        "num_procedures",
        "num_medications",
        "number_outpatient",
        "number_emergency",
        "number_inpatient",
        "number_diagnoses",
        "insulin",
        "diabetesMed"
    ]

    df_model = df_eligible[
        ["patient_nbr", "readmitted_30"] + predictors
    ].copy()
```

```
    print("Encounters:", len(df_model))
    print("Predictors:", len(predictors))
    df_model.head()
    return df_model, predictors


@app.cell
def _(df_model):
    from sklearn.model_selection import GroupShuffleSplit

    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.20,
        random_state=42
    )

    train_idx, test_idx = next(
        splitter.split(
            df_model,
            df_model["readmitted_30"],
            groups=df_model["patient_nbr"]
        )
    )

    train_df = df_model.iloc[train_idx].copy()
    test_df = df_model.iloc[test_idx].copy()

    print("Training encounters:", len(train_df))
    print("Testing encounters:", len(test_df))

    print(
        "Patients in both sets:",
        len(
            set(train_df["patient_nbr"]) &
            set(test_df["patient_nbr"])
        )
    )
    return test_df, train_df


@app.cell
def _(test_df, train_df):
    print("Training readmission counts:")
    print(train_df["readmitted_30"].value_counts())

    print("\nTraining readmission rate:")
    print(round(train_df["readmitted_30"].mean() * 100, 2), "%")

    print("\nTesting readmission counts:")
    print(test_df["readmitted_30"].value_counts())
```

```
    print("\nTesting readmission rate:")
    print(round(test_df["readmitted_30"].mean() * 100, 2), "%")
    return


@app.cell
def _(train_df):
    baseline_probability = train_df["readmitted_30"].mean()

    print("Baseline readmission probability:")
    print(round(baseline_probability, 4))

    print("\nBaseline readmission percentage:")
    print(round(baseline_probability * 100, 2), "%")
    return (baseline_probability,)


@app.cell
def _(np, predictors, test_df, train_df):
    X_train = train_df[predictors].replace("?", np.nan)
    y_train = train_df["readmitted_30"]

    X_test = test_df[predictors].replace("?", np.nan)
    y_test = test_df["readmitted_30"]

    print("Training predictors:", X_train.shape)
    print("Training outcomes:", y_train.shape)
    print("Testing predictors:", X_test.shape)
    print("Testing outcomes:", y_test.shape)
    return X_test, X_train, y_test, y_train


@app.cell
def _():
    numeric_features = [
         "time_in_hospital",
         "num_lab_procedures",
         "num_procedures",
         "num_medications",
         "number_outpatient",
         "number_emergency",
         "number_inpatient",
         "number_diagnoses"
    ]

    categorical_features = [
        "age",
        "gender",
        "race",
```

```
        "admission_type_id",
        "insulin",
        "diabetesMed"
    ]

    print("Numerical predictors:", len(numeric_features))
    print("Categorical predictors:", len(categorical_features))
    print("Total predictors:", len(numeric_features) +
len(categorical_features))
    return categorical_features, numeric_features


@app.cell
def _(categorical_features, numeric_features):
    from sklearn.compose import ColumnTransformer
    from sklearn.pipeline import Pipeline
    from sklearn.impute import SimpleImputer
    from sklearn.preprocessing import StandardScaler, OneHotEncoder

    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])

    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ])

    preprocessor = ColumnTransformer([
        ("numeric", numeric_pipeline, numeric_features),
        ("categorical", categorical_pipeline, categorical_features)
    ])

    print("Preprocessing pipeline created successfully!")
    return (
        ColumnTransformer,
        Pipeline,
        categorical_pipeline,
        numeric_pipeline,
        preprocessor,
    )


@app.cell
def _(Pipeline, preprocessor):
    from sklearn.linear_model import LogisticRegression

    logistic_model = Pipeline([
        ("preprocessing", preprocessor),
```

```
        ("classifier", LogisticRegression(
            max_iter=1000,
            random_state=42
        ))
    ])

    print("Logistic regression model created successfully!")
    return (logistic_model,)


@app.cell
def _(X_train, logistic_model, train_df, y_train):
    from sklearn.model_selection import GroupKFold, GridSearchCV

    cv = GroupKFold(n_splits=3)

    logistic_search = GridSearchCV(
        estimator=logistic_model,
        param_grid={
            "classifier__C": [0.1, 1.0, 10.0]
        },
        scoring="roc_auc",
        cv=cv,
        n_jobs=1
    )

    logistic_search.fit(
        X_train,
        y_train,


        groups=train_df["patient_nbr"]
    )

    print("Best C:", logistic_search.best_params_)
    print("Best cross-validation AUROC:",
          round(logistic_search.best_score_, 4))
    return GridSearchCV, GroupKFold, logistic_search


@app.cell
def _(logistic_search, pd):
    logistic_cv_results = pd.DataFrame(
        logistic_search.cv_results_
    )[[
        "param_classifier__C",
        "mean_test_score",
        "std_test_score"
    ]]
```

```
    logistic_cv_results.columns = [
        "C",
        "Mean AUROC",
        "AUROC Standard Deviation"
    ]

    logistic_cv_results
    return


@app.cell
def _(Pipeline, preprocessor):
    from sklearn.ensemble import RandomForestClassifier

    forest_model = Pipeline([
        ("preprocessing", preprocessor),
        ("classifier", RandomForestClassifier(
            n_estimators=100,
            random_state=42,
            n_jobs=1
        ))
    ])

    print("Random forest model created successfully!")
    return RandomForestClassifier, forest_model


@app.cell
def _(GridSearchCV, GroupKFold, X_train, forest_model, train_df,
y_train):
    forest_search = GridSearchCV(
        estimator=forest_model,
        param_grid={
            "classifier__max_depth": [5, 10],
            "classifier__min_samples_leaf": [10, 30]
        },
        scoring="roc_auc",
        cv=GroupKFold(n_splits=3),
        n_jobs=1
    )

    forest_search.fit(
        X_train,
        y_train,
        groups=train_df["patient_nbr"]
    )

    print("Best settings:", forest_search.best_params_)
    print(
        "Best cross-validation AUROC:",
```

```
        round(forest_search.best_score_, 4)
    )
    return (forest_search,)


@app.cell
def _(forest_search, pd):
    forest_cv_results = pd.DataFrame(
        forest_search.cv_results_
    )[[
        "param_classifier__max_depth",
        "param_classifier__min_samples_leaf",
        "mean_test_score",
        "std_test_score"
    ]]

    forest_cv_results.columns = [
        "Max Depth",
        "Min Samples per Leaf",
        "Mean AUROC",
        "AUROC Standard Deviation"
    ]

    forest_cv_results
    return


@app.cell
def _(forest_search, logistic_search, pd):
    model_comparison = pd.DataFrame({
        "Model": ["Logistic Regression", "Random Forest"],
        "Mean CV AUROC": [
            logistic_search.best_score_,
            forest_search.best_score_
        ],
        "CV Standard Deviation": [
            logistic_search.cv_results_["std_test_score"][
                logistic_search.best_index_
            ],
            forest_search.cv_results_["std_test_score"][
                forest_search.best_index_


             ]
         ]
    })

    model_comparison
    return
```

```
@app.cell
def _(GroupKFold, X_train, forest_search, train_df, y_train):
    from sklearn.model_selection import cross_val_predict
    from sklearn.metrics import confusion_matrix, recall_score,
precision_score

    cv_probabilities = cross_val_predict(
        forest_search.best_estimator_,
        X_train,
        y_train,
        groups=train_df["patient_nbr"],
        cv=GroupKFold(n_splits=3),
        method="predict_proba",
        n_jobs=1
    )[:, 1]

    threshold = 0.15
    cv_predictions = (cv_probabilities >= threshold).astype(int)

    print("Threshold:", threshold)
    print("Confusion matrix:")
    print(confusion_matrix(y_train, cv_predictions))
    print("Sensitivity:", round(recall_score(y_train, cv_predictions),
4))
    print("Precision:", round(precision_score(y_train, cv_predictions,
zero_division=0), 4))
    return confusion_matrix, cv_probabilities, precision_score,
recall_score


@app.cell
def _(cv_probabilities, pd, precision_score, recall_score, y_train):
    threshold_results = []

    for t in [0.10, 0.12, 0.15, 0.20]:
        predictions = (cv_probabilities >= t).astype(int)

        threshold_results.append({
            "Threshold": t,
            "Sensitivity": recall_score(y_train, predictions),
            "Precision": precision_score(
                y_train, predictions, zero_division=0
            ),
            "Flagged (%)": predictions.mean() * 100
        })

    pd.DataFrame(threshold_results).round(3)
    return
```

```
@app.cell
def _():
    final_threshold = 0.12

    print("Selected model: Random Forest")
    print("Selected threshold:", final_threshold)
    print("Maximum tree depth:", 10)
    print("Minimum samples per leaf:", 30)
    print("Selection based on training cross-validation only.")
    return (final_threshold,)


@app.cell
def _(X_test, final_threshold, forest_search, logistic_search,
y_test):
    # Use the best models selected during cross-validation
    best_logistic = logistic_search.best_estimator_
    best_forest = forest_search.best_estimator_

    # Generate predicted probabilities on the held-out test set
    logistic_probabilities = best_logistic.predict_proba(X_test)[:, 1]
    forest_probabilities = best_forest.predict_proba(X_test)[:, 1]

    # Apply our previously selected 12% threshold
    logistic_predictions = (logistic_probabilities >=
final_threshold).astype(int)
    forest_predictions = (forest_probabilities >=
final_threshold).astype(int)

    print("Test encounters:", len(y_test))
    print("Logistic regression predictions:",
len(logistic_predictions))
    print("Random forest predictions:", len(forest_predictions))
    print("Threshold used:", final_threshold)
    return (
        best_forest,
        forest_predictions,
        forest_probabilities,
        logistic_predictions,
        logistic_probabilities,
    )


@app.cell
def _(
    baseline_probability,
    forest_probabilities,
    logistic_probabilities,
    np,
```

```
    pd,
    y_test,
):
    from sklearn.metrics import roc_auc_score, average_precision_score

    # Baseline predicts the same probability for every encounter
    baseline_probs = np.full(len(y_test), baseline_probability)



    # Compare all three models
    evaluation_results = pd.DataFrame({
        "Model": ["Baseline", "Logistic Regression", "Random Forest"],
        "AUROC": [
            roc_auc_score(y_test, baseline_probs),
            roc_auc_score(y_test, logistic_probabilities),
            roc_auc_score(y_test, forest_probabilities)
        ],
        "AUPRC": [
            average_precision_score(y_test, baseline_probs),
            average_precision_score(y_test, logistic_probabilities),
            average_precision_score(y_test, forest_probabilities)
        ]
    })

    evaluation_results.round(4)
    return baseline_probs, roc_auc_score


@app.cell
def _(forest_probabilities, logistic_probabilities, plt, y_test):
    from sklearn.metrics import RocCurveDisplay

    fig_roc, ax_roc = plt.subplots(figsize=(7, 5))

    RocCurveDisplay.from_predictions(
        y_test,
        logistic_probabilities,
        name="Logistic Regression",
        ax=ax_roc
    )

    RocCurveDisplay.from_predictions(
        y_test,
        forest_probabilities,
        name="Random Forest",
        ax=ax_roc
    )

    ax_roc.plot([0, 1], [0, 1], "k--", label="Random Guessing")
```

```
    ax_roc.set_title("ROC Curve: 30-Day Readmission Prediction")
    ax_roc.legend(loc="lower right")

    fig_roc
    return


@app.cell
def _(forest_probabilities, logistic_probabilities, plt, y_test):
    from sklearn.metrics import PrecisionRecallDisplay

    fig_pr, ax_pr = plt.subplots(figsize=(7, 5))

    PrecisionRecallDisplay.from_predictions(
        y_test,
        logistic_probabilities,
        name="Logistic Regression",
        ax=ax_pr
    )

    PrecisionRecallDisplay.from_predictions(
        y_test,
        forest_probabilities,
        name="Random Forest",
        ax=ax_pr
    )

    ax_pr.axhline(
        y_test.mean(),
        color="black",
        linestyle="--",
        label="Baseline"
    )

    ax_pr.set_title("Precision-Recall Curve: 30-Day Readmission")
    ax_pr.legend(loc="upper right")

    fig_pr
    return


@app.cell
def _(confusion_matrix, forest_predictions, logistic_predictions, pd,
y_test):
    classification_results = []

    for model_name, test_preds in [
        ("Logistic Regression", logistic_predictions),
        ("Random Forest", forest_predictions)
```

```
    ]:
        tn, fp, fn, tp = confusion_matrix(
            y_test, test_preds
        ).ravel()

        classification_results.append({
            "Model": model_name,
            "True Positives": tp,
            "False Positives": fp,
            "True Negatives": tn,
            "False Negatives": fn,
            "Sensitivity": tp / (tp + fn),
            "Specificity": tn / (tn + fp),
            "Precision (PPV)": tp / (tp + fp) if (tp + fp) > 0 else 0
        })

    classification_table = pd.DataFrame(classification_results)

    classification_table.round(4)
    return


@app.cell
def _(
    baseline_probs,
    forest_probabilities,
    logistic_probabilities,
    pd,
    y_test,
):
    from sklearn.metrics import brier_score_loss

    brier_results = pd.DataFrame({
        "Model": ["Baseline", "Logistic Regression", "Random Forest"],
        "Brier Score": [
            brier_score_loss(y_test, baseline_probs),
            brier_score_loss(y_test, logistic_probabilities),
            brier_score_loss(y_test, forest_probabilities)
        ]
    })

    brier_results.round(4)
    return


@app.cell
def _(forest_probabilities, logistic_probabilities, plt, y_test):
    from sklearn.calibration import CalibrationDisplay

    fig_cal, ax_cal = plt.subplots(figsize=(7, 5))
```

```
    CalibrationDisplay.from_predictions(
        y_test,
        logistic_probabilities,
        n_bins=10,
        strategy="quantile",
        name="Logistic Regression",
        ax=ax_cal
    )

    CalibrationDisplay.from_predictions(
        y_test,
        forest_probabilities,
        n_bins=10,
        strategy="quantile",
        name="Random Forest",
        ax=ax_cal
    )

    ax_cal.plot(
        [0, 1], [0, 1],
        "k--",
        label="Perfect Calibration"
    )

    ax_cal.set_title("Calibration Plot: 30-Day Readmission")
    ax_cal.legend()

    fig_cal
    return


@app.cell
def _(X_test, best_forest, pd, y_test):
    from sklearn.inspection import permutation_importance

    importance_result = permutation_importance(
        best_forest,
        X_test,
        y_test,
        scoring="roc_auc",
        n_repeats=3,
        random_state=42,
        n_jobs=1
    )

    importance_table = pd.DataFrame({
        "Predictor": X_test.columns,
        "Importance": importance_result.importances_mean,
        "Standard Deviation": importance_result.importances_std
```

```
    }).sort_values("Importance", ascending=False)

    importance_table.round(4)
    return (importance_table,)


@app.cell
def _(importance_table, plt):
    top_importance =
importance_table.head(10).sort_values("Importance")

    fig_imp, ax_imp = plt.subplots(figsize=(8, 5))

    ax_imp.barh(
        top_importance["Predictor"],
        top_importance["Importance"],
        xerr=top_importance["Standard Deviation"],
        capsize=3
    )

    ax_imp.set_xlabel("Decrease in AUROC When Predictor Is Shuffled")
    ax_imp.set_title("Random Forest: Top 10 Predictor Importances")

    fig_imp.tight_layout()
    fig_imp
    return


@app.cell
def _(X_test, forest_probabilities, np, y_test):


    # Find the highest-risk and lowest-risk test encounters
    high_risk_idx = np.argmax(forest_probabilities)
    low_risk_idx = np.argmin(forest_probabilities)

    # Display their clinical information and predictions
    case_indices = [high_risk_idx, low_risk_idx]

    case_table = X_test.iloc[case_indices].copy()

    case_table["Predicted Readmission Probability"] = (
        forest_probabilities[case_indices]
    )

    case_table["Actual Readmission"] = (
        y_test.iloc[case_indices].values
    )

    case_table.index = ["Highest Predicted Risk", "Lowest Predicted
```

```
Risk"]

    case_table.T
    return (case_indices,)


@app.cell
def _(X_test, best_forest, case_indices, pd):
    case_explanation = X_test.iloc[case_indices].copy()

    original_risks = best_forest.predict_proba(
        case_explanation
    )[:, 1]

    # Set previous inpatient visits to zero for both cases
    modified_cases = case_explanation.copy()
    modified_cases["number_inpatient"] = 0

    modified_risks = best_forest.predict_proba(
        modified_cases
    )[:, 1]

    case_explanation_results = pd.DataFrame({
        "Case": ["Highest Predicted Risk", "Lowest Predicted Risk"],
        "Previous Inpatient Visits":
case_explanation["number_inpatient"].values,
        "Original Risk (%)": original_risks * 100,
        "Risk With 0 Prior Inpatient Visits (%)": modified_risks *
100,
        "Change (Percentage Points)": (
            modified_risks - original_risks
        ) * 100
    })

    case_explanation_results.round(2)
    return


@app.cell
def _(
    X_test,
    best_forest,
    forest_predictions,
    forest_probabilities,
    np,
    y_test,
):
    # Find an incorrect prediction (false positive)
    false_positive_positions = np.where(
        (forest_predictions == 1) & (y_test.to_numpy() == 0)
```

```
    )[0]

    # Select the false positive with the highest predicted risk
    i = false_positive_positions[
        np.argmax(forest_probabilities[false_positive_positions])
    ]

    # Display the actual outcome and predicted risk
    print("INDIVIDUAL INCORRECT PREDICTION")
    print("-------------------------------")
    print("Actual 30-day readmission:", int(y_test.iloc[i]))
    print("Predicted 30-day readmission:", int(forest_predictions[i]))
    print("Predicted probability:", round(forest_probabilities[i] *
100, 2), "%")
    print("Age group:", X_test.iloc[i]["age"])
    print("Previous inpatient visits:", X_test.iloc[i]
["number_inpatient"])
    print("Previous emergency visits:", X_test.iloc[i]
["number_emergency"])

    # Examine how one predictor affects the model's prediction
    modified_case = X_test.iloc[[i]].copy()
    modified_case["number_inpatient"] = 0

    modified_probability = best_forest.predict_proba(modified_case)[0,
1]

    print(
        "Predicted probability with 0 previous inpatient visits:",
        round(modified_probability * 100, 2),
        "%"
    )
    return


@app.cell
def _(X_test, forest_probabilities, pd, roc_auc_score, y_test):
    age_results = []

    for age_group in sorted(X_test["age"].dropna().unique()):
        mask = X_test["age"] == age_group

        actual = y_test[mask]
        predicted = forest_probabilities[mask.to_numpy()]

        if len(actual) > 0 and actual.nunique() == 2:
            age_results.append({
                "Age Group": age_group,
                "Encounters": len(actual),
                "Readmission Rate (%)": actual.mean() * 100,
```

```
                "AUROC": roc_auc_score(actual, predicted)
            })

    age_subgroup_results = pd.DataFrame(age_results)

    age_subgroup_results.round(4)
    return (age_subgroup_results,)


@app.cell
def _(age_subgroup_results, plt):
    fig_age, ax_age = plt.subplots(figsize=(9, 5))

    ax_age.bar(
        age_subgroup_results["Age Group"],
        age_subgroup_results["AUROC"]
    )

    ax_age.axhline(
        y=0.5,
        color="black",
        linestyle="--",
        label="Random Guessing"
    )

    ax_age.set_title("Random Forest AUROC by Age Group")
    ax_age.set_xlabel("Age Group")
    ax_age.set_ylabel("AUROC")
    ax_age.set_ylim(0, 1)

    ax_age.tick_params(axis="x", rotation=45)
    ax_age.legend()

    fig_age.tight_layout()
    fig_age
    return


@app.cell
def _(X_test, forest_probabilities, pd, roc_auc_score, y_test):
    # Restrict the test data to encounters aged 40 and older


    older_age_groups = [
        "[40-50)", "[50-60)", "[60-70)",
        "[70-80)", "[80-90)", "[90-100)"
    ]

    older_mask = X_test["age"].isin(older_age_groups)
```

```
    older_actual = y_test[older_mask]
    older_probabilities = forest_probabilities[older_mask.to_numpy()]

    older_results = pd.DataFrame({
        "Population": ["All Test Encounters", "Age 40 and Older"],
        "Encounters": [len(y_test), len(older_actual)],
        "Readmission Rate (%)": [
            y_test.mean() * 100,
            older_actual.mean() * 100
        ],
        "AUROC": [
            roc_auc_score(y_test, forest_probabilities),
            roc_auc_score(older_actual, older_probabilities)
        ]
    })

    older_results.round(4)
    return


@app.cell
def _(forest_probabilities, np, pd, y_test):
    error_results = pd.DataFrame({
        "Actual Readmission": y_test.values,
        "Predicted Probability": forest_probabilities
    })

    error_results["Predicted Readmission"] = (
        error_results["Predicted Probability"] >= 0.12
    ).astype(int)

    error_results["Prediction Type"] = np.select(
        [
            (error_results["Actual Readmission"] == 1) &
            (error_results["Predicted Readmission"] == 1),

             (error_results["Actual Readmission"] == 0) &
             (error_results["Predicted Readmission"] == 0),

             (error_results["Actual Readmission"] == 0) &
             (error_results["Predicted Readmission"] == 1),

             (error_results["Actual Readmission"] == 1) &
             (error_results["Predicted Readmission"] == 0)
        ],
        [
             "True Positive",
             "True Negative",
             "False Positive",
             "False Negative"
```

```
        ],
        default="Unknown"
    )

    error_results["Prediction Type"].value_counts()
    return (error_results,)


@app.cell
def _(error_results, y_test):
    error_summary = (
        error_results["Prediction Type"]
        .value_counts()
        .rename_axis("Prediction Type")
        .reset_index(name="Encounters")
    )

    error_summary["Percentage of Test Encounters"] = (
        error_summary["Encounters"] / len(y_test) * 100
    )

    error_summary.round(2)
    return


@app.cell
def _(predictors):
    # Sensitivity analysis: remove previous inpatient visits

    reduced_predictors = [
        p for p in predictors if p != "number_inpatient"
    ]

    print("Original predictors:", len(predictors))
    print("Reduced predictors:", len(reduced_predictors))
    print("Removed predictor: number_inpatient")
    return (reduced_predictors,)


@app.cell
def _(
    ColumnTransformer,
    Pipeline,
    RandomForestClassifier,
    categorical_features,
    categorical_pipeline,
    numeric_features,
    numeric_pipeline,
):
    # Create preprocessing without number_inpatient
```

```
    reduced_numeric_features = [


        p for p in numeric_features if p != "number_inpatient"
    ]

    reduced_preprocessor = ColumnTransformer([
        ("numeric", numeric_pipeline, reduced_numeric_features),
        ("categorical", categorical_pipeline, categorical_features)
    ])

    # Use the same random forest settings as the original model
    reduced_forest = Pipeline([
        ("preprocessing", reduced_preprocessor),
        ("classifier", RandomForestClassifier(
            n_estimators=100,
            max_depth=10,
            min_samples_leaf=30,
            random_state=42,
            n_jobs=1
        ))
    ])

    print("Reduced random forest created successfully!")
    return (reduced_forest,)


@app.cell
def _(
    GroupKFold,
    X_train,
    reduced_forest,
    reduced_predictors,
    train_df,
    y_train,
):
    # Compare the reduced model using patient-level cross-validation

    from sklearn.model_selection import cross_val_score

    reduced_cv_scores = cross_val_score(
        reduced_forest,
        X_train[reduced_predictors],
        y_train,
        groups=train_df["patient_nbr"],
        cv=GroupKFold(n_splits=3),
        scoring="roc_auc",
        n_jobs=1
    )
```

```
    print("Reduced model mean CV AUROC:",
          round(reduced_cv_scores.mean(), 4))

    print("Reduced model CV standard deviation:",
          round(reduced_cv_scores.std(), 4))
    return (reduced_cv_scores,)


@app.cell
def _(reduced_cv_scores):
    # Sensitivity analysis comparison

    original_auroc = 0.6402
    reduced_auroc = reduced_cv_scores.mean()

    print("SENSITIVITY ANALYSIS")
    print("-----------------------------")
    print(f"Original model CV AUROC: {original_auroc:.4f}")
    print(f"Reduced model CV AUROC: {reduced_auroc:.4f}")
    print(f"AUROC difference:        {original_auroc -
reduced_auroc:.4f}")

    print(
        "\nRemoving number_inpatient reduced model discrimination, "
        "suggesting that previous inpatient utilization is an "
        "important predictor of 30-day readmission."
    )
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    ## Final Project Summary

    ### Research Question

    Among hospitalized patients with diabetes, how accurately can
clinical information available at discharge predict hospital
readmission within 30 days?

    ### Dataset and Methods

    We analyzed the Diabetes 130-US Hospitals dataset, containing
101,766 hospital encounters. After excluding encounters with discharge
dispositions indicating death or hospice care, 99,343 encounters from
69,990 unique patients remained eligible. We used 14 clinical and
demographic predictors available at discharge to develop logistic
regression and random forest models. Patients were separated between
```

training and testing to prevent information leakage. Hyperparameter tuning was performed using cross-validation on the training data.

Main Findings

The random forest slightly outperformed logistic regression on the held-out test set, achieving an AUROC of 0.6349 compared with 0.6306. Both models performed better than the baseline AUROC of 0.5000. The random forest achieved an AUPRC of 0.1957 and a Brier score of 0.0965. At the selected 12% classification threshold, it identified approximately 50% of recorded 30-day readmissions, although many encounters were incorrectly flagged.

Previous inpatient hospitalizations were the most influential predictor. Age subgroup analysis suggested differences in model performance across age groups. An analysis restricted to patients aged 40 and older produced a test AUROC of 0.6240. An additional sensitivity analysis using patient-level cross-validation on the training data showed that removing previous inpatient hospitalizations (`number_inpatient`) reduced the random forest's AUROC from 0.6402 to 0.5953, highlighting the importance of prior hospital utilization in predicting 30-day readmission. An individual prediction analysis identified a false positive with a predicted 30-day readmission risk of 38.26%, despite no recorded readmission within 30 days. Setting previous inpatient visits to zero reduced the model's predicted risk to 17.05%, illustrating the influence of prior hospital utilization without establishing causation.

Clinical Interpretation and Limitations

The models demonstrated modest predictive ability, suggesting that routinely collected clinical information contains some useful information about readmission risk. However, the relatively low sensitivity and positive predictive value limit their clinical usefulness. The dataset is historical, contains substantial missing information in several variables, and may not capture readmissions outside participating hospitals. The results should therefore be interpreted as an exploratory prediction study.

Conclusion

Machine learning models can identify some patterns associated with 30-day hospital readmission among patients with diabetes, but the predictive performance achieved in this project is insufficient to support independent clinical decision-making. Further validation, improved clinical data, and additional research would be necessary before considering clinical implementation.

```
    """)
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""

    """)
    return


if __name__ == "__main__":
    app.run()
```

Upload Uploaded 1 file. Click to clear files.


```
[ 2 Items
  0: 101766
  1: 50
]
```


Search...

encounter_id int64	patient_nbr int64	race str	gender str	age str	weight str	admission_type_id int64
2,278,392	8,222,157	Caucasian	Female	[0-10)	?	
149,190	55,629,189	Caucasian	Female	[10-20)	?	
64,410	86,047,875	AfricanAmerican	Female	[20-30)	?	
500,364	82,442,376	Caucasian	Male	[30-40)	?	
16,680	42,519,267	Caucasian	Male	[40-50)	?	

5 rows, 50 columns *No selection*

Search...

readmitted str	count int64
NO	54,864
>30	35,545
<30	11,357

3 rows, 1 column *No selection*

Search...

readmitted_30 int32	count int64
0	90,409
1	11,357

2 rows, 1 column *No selection*

Search...

discharge_disposition_id int64	count int64
1	60,234
2	2,128
3	13,954
4	815
5	1,184
6	12,902
7	623
8	108
9	21
10	6

26 rows, 1 column *No selection*

Search...

discharge_disposition_id int64	count int64
11	1,642
13	399
14	372
19	8
20	2
21	0

6 rows, 1 column *No selection*

```
Original encounters: 101766
Excluded encounters: 2423
Eligible encounters: 99343
Unique patients: 69990
```

```
Readmission counts:
readmitted_30
0    88029
```

```
1    11314
Name: count, dtype: int64

30-day readmission rate:
11.39 %
```

Search...

Index0 str	value int64
weight	96,218
max_glu_serum	94,191
A1Cresult	82,509
medical_specialty	48,616
payer_code	39,398
race	2,234
diag_3	1,419
diag_2	356
diag_1	20

9 rows, 1 column *No selection*

```
Exact duplicate rows: 0
Duplicate encounter IDs: 0
Patients with multiple encounters: 16341
```

```
Available columns:
encounter_id
patient_nbr
race
gender
age
weight
admission_type_id
discharge_disposition_id
admission_source_id
time_in_hospital
payer_code
medical_specialty
num_lab_procedures
num_procedures
num_medications
number_outpatient
number_emergency
number_inpatient
diag_1
diag_2
diag_3
number_diagnoses
max_glu_serum
A1Cresult
metformin
repaglinide
nateglinide
chlorpropamide
glimepiride
acetohexamide
glipizide
glyburide
tolbutamide
pioglitazone
rosiglitazone
acarbose
miglitol
troglitazone
tolazamide
examide
citoglipton
insulin
glyburide-metformin
glipizide-metformin
glimepiride-pioglitazone
metformin-rosiglitazone
metformin-pioglitazone
change
diabetesMed
readmitted
readmitted_30
```

Search...

Index0 int64	patient_nbr int64	readmitted_30 int32	age str	gender str	race str	admission_type_ int64
0	8,222,157	0	[0-10)	Female	Caucasian	
1	55,629,189	0	[10-20)	Female	Caucasian	
2	86,047,875	0	[20-30)	Female	AfricanAmerican	
3	82,442,376	0	[30-40)	Male	Caucasian	
4	42,519,267	0	[40-50)	Male	Caucasian	

5 rows, 16 columns *No selection*

```
Encounters: 99343
Predictors: 14
```

```
Training encounters: 79541
Testing encounters: 19802
Patients in both sets: 0
```

```
Training readmission counts:
readmitted_30
0    70443
1     9098
Name: count, dtype: int64

Training readmission rate:
11.44 %

Testing readmission counts:
readmitted_30
0    17586
1     2216
Name: count, dtype: int64

Testing readmission rate:
11.19 %
```

```
Baseline readmission probability:
0.1144

Baseline readmission percentage:
11.44 %
```

```
Training predictors: (79541, 14)
Training outcomes: (79541,)
Testing predictors: (19802, 14)
Testing outcomes: (19802,)
```

```
Numerical predictors: 8
Categorical predictors: 6
Total predictors: 14
```

```
Preprocessing pipeline created successfully!
```

```
Logistic regression model created successfully!
```

```
Best C: {'classifier__C': 0.1}
Best cross-validation AUROC: 0.6379
```

Search...

C float64	Mean AUROC float64	AUROC Standard Deviation float64	
0.1	0.6378625434360955	0.0036654249504289626	
1.0	0.6374636578986160	0.0034780252236517600	
10.0	0.6373996958522990	0.0034511331122262910	

3 rows, 3 columns *No selection*

```
Random forest model created successfully!
```

```
Best settings: {'classifier__max_depth': 10, 'classifier__min_samples_leaf': 30}
Best cross-validation AUROC: 0.6402
```

Search...

Max Depth int32	Min Samples per Leaf int32	Mean AUROC float64	AUROC Standard Deviation float64	
5	10	0.6366165398354687	0.0044523987915394470	
5	30	0.6361306304520026	0.0044074504001454864	
10	10	0.6389025310035480	0.0041988980621818950	
10	30	0.6401550657181908	0.0046483659675389580	

4 rows, 4 columns *No selection*

Search...

Model str	Mean CV AUROC float64	CV Standard Deviation float64	
Logistic Regression	0.6378625434360955	0.0036654249504289626	
Random Forest	0.6401550657181908	0.0046483659675389580	

2 rows, 3 columns *No selection*

```
Threshold: 0.15
Confusion matrix:
[[60670  9773]
 [ 6436  2662]]
Sensitivity: 0.2926
Precision: 0.2141
```

Search...

Threshold float64	Sensitivity float64	Precision float64	Flagged (%) float64	
0.10	0.735	0.145	58.003	
0.12	0.498	0.172	33.026	
0.15	0.293	0.214	15.633	
0.20	0.115	0.289	4.570	

4 rows, 4 columns *No selection*

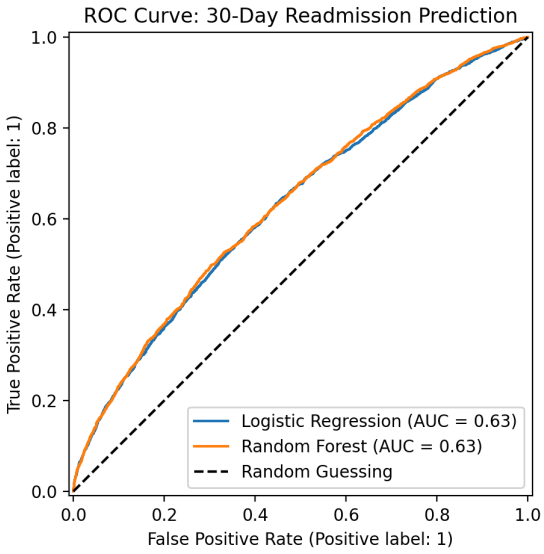
```
Selected model: Random Forest
Selected threshold: 0.12
Maximum tree depth: 10
Minimum samples per leaf: 30
Selection based on training cross-validation only.
```

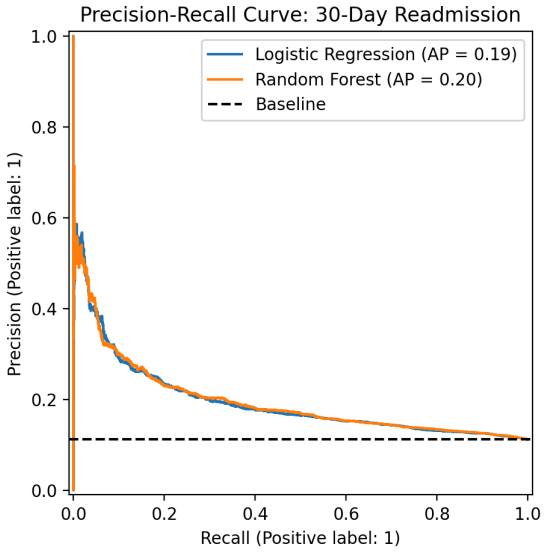
```
Test encounters: 19802
Logistic regression predictions: 19802
Random forest predictions: 19802
Threshold used: 0.12
```

Search...

Model str	AUROC float64	AUPRC float64	
Baseline	0.5000	0.1119	
Logistic Regression	0.6306	0.1938	
Random Forest	0.6349	0.1957	

3 rows, 3 columns *No selection*





Search...

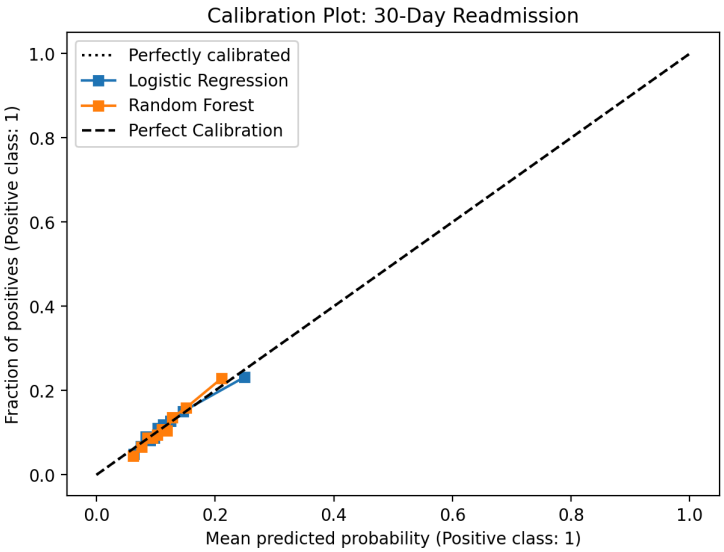
Model str	True Positives int64	False Positives int64	True Negatives int64	False Negatives int64	Sensitivi float64
Logistic Regression	969	4,601	12,985	1,247	
Random Forest	1,107	5,402	12,184	1,109	

2 rows, 8 columns *No selection*

Search...

Model str	Brier Score float64
Baseline	0.0994
Logistic Regression	0.0967
Random Forest	0.0965

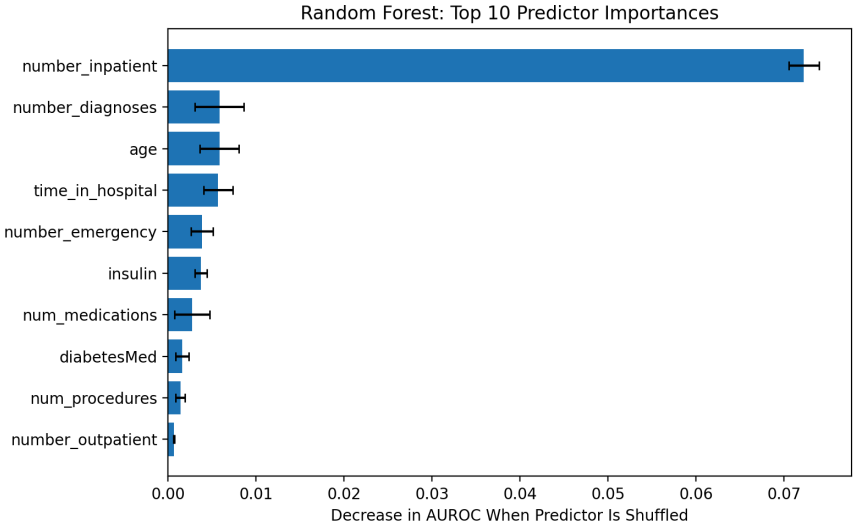
3 rows, 2 columns *No selection*



Search...

Index0 int64	Predictor str unique: 14	Importance float64	Standard Deviation float64
10	number_inpatient	0.0722	0.0017
11	number_diagnoses	0.0058	0.0028
0	age	0.0058	0.0022
4	time_in_hospital	0.0057	0.0017
9	number_emergency	0.0039	0.0012
12	insulin	0.0037	0.0007
7	num_medications	0.0027	0.0020
13	diabetesMed	0.0016	0.0007
6	num_procedures	0.0014	0.0005
8	number_outpatient	0.0007	0.0001

14 rows, 3 columns *No selection*



Search...

Index0 str	Highest Predicted Risk object unique:	Lowest Predicted Risk object unique:	
age	[30-40)	[50-60)	
gender	Female	Male	
race	Caucasian	Caucasian	
admission_type_id	3	2	
time_in_hospital	1	1	
num_lab_procedures	44	18	
num_procedures	1	5	
num_medications	14	12	
number_outpatient	0	0	
number_emergency	12	0	

16 rows, 2 columns *No selection*

Search...

Case str	Previous Inpatient Visits int64	Original Risk (%) float64	Risk With 0 Prior Inpatient Visits (%) float64
Highest Predicted Risk	10	38.26	17.05
Lowest Predicted Risk	0	3.66	3.66

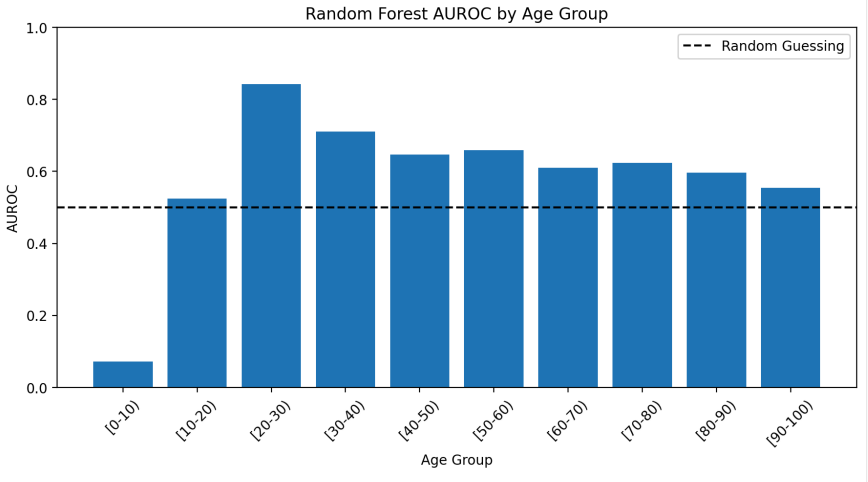
2 rows, 5 columns *No selection*

```
INDIVIDUAL INCORRECT PREDICTION
--------------------------------
Actual 30-day readmission: 0
Predicted 30-day readmission: 1
Predicted probability: 38.26 %
Age group: [30-40)
Previous inpatient visits: 10
Previous emergency visits: 12
Predicted probability with 0 previous inpatient visits: 17.05 %
```

Search...

Age Group str	Encounters int64	Readmission Rate (%) float64	AUROC float64	
[0-10)	29	3.4483	0.0714	
[10-20)	138	6.5217	0.5245	
[20-30)	388	15.4639	0.8425	
[30-40)	729	10.5624	0.7108	
[40-50)	1,909	10.5815	0.6467	
[50-60)	3,469	10.2623	0.6585	
[60-70)	4,266	10.8767	0.6101	
[70-80)	5,040	11.6468	0.6241	
[80-90)	3,287	11.8649	0.5963	
[90-100)	547	12.7971	0.5548	

10 rows, 4 columns *No selection*



Search...

Population str	Encounters int64	Readmission Rate (%) float64	AUROC float64	
All Test Encounters	19,802	11.1908	0.6349	
Age 40 and Older	18,518	11.1729	0.6240	

2 rows, 4 columns *No selection*

Search...

Prediction Type str	count int64	
True Negative	12,184	
False Positive	5,402	
False Negative	1,109	
True Positive	1,107	

4 rows, 1 column *No selection*

Search...

Prediction Type str	Encounters int64	Percentage of Test Encounters float64	
True Negative	12,184	61.53	
False Positive	5,402	27.28	
False Negative	1,109	5.60	
True Positive	1,107	5.59	

4 rows, 3 columns *No selection*

```
Original predictors: 14
Reduced predictors: 13
Removed predictor: number_inpatient
```

```
Reduced random forest created successfully!
```

```
Reduced model mean CV AUROC: 0.5953
Reduced model CV standard deviation: 0.003
```

```
NSITIVITY ANALYSIS
----------------------------
iginal model CV AUROC: 0.6402
duced model CV AUROC: 0.5953
ROC difference:        0.0449

moving number_inpatient reduced model discrimination, suggesting that previous inpatient utilization is
```

Final Project Summary

Research Question

Among hospitalized patients with diabetes, how accurately can clinical information available at discharge predict hospital readmission within 30 days?

Dataset and Methods

We analyzed the Diabetes 130-US Hospitals dataset, containing 101,766 hospital encounters. After excluding encounters with discharge dispositions indicating death or hospice care, 99,343 encounters from 69,990 unique patients remained eligible. We used 14 clinical and demographic predictors available at discharge to develop logistic regression and random forest models. Patients were separated between training and testing to prevent information leakage. Hyperparameter tuning was performed using cross-validation on the training data.

Main Findings

The random forest slightly outperformed logistic regression on the held-out test set, achieving an AUROC of 0.6349 compared with 0.6306. Both models performed better than the baseline AUROC of 0.5000. The random forest achieved an AUPRC of 0.1957 and a Brier score of 0.0965. At the selected 12% classification threshold, it identified approximately 50% of recorded 30-day readmissions, although many encounters were incorrectly flagged.

Previous inpatient hospitalizations were the most influential predictor. Age subgroup analysis suggested differences in model performance across age groups. An analysis restricted to patients aged 40 and older produced a test AUROC of 0.6240. An additional sensitivity analysis using patient-level cross-validation on the training data showed that removing previous inpatient hospitalizations (`number_inpatient`) reduced the random forest's AUROC from 0.6402

to 0.5953, highlighting the importance of prior hospital utilization in predicting 30-day readmission. An individual prediction analysis identified a false positive with a predicted 30-day readmission risk of 38.26%, despite no recorded readmission within 30 days. Setting previous inpatient visits to zero reduced the model's predicted risk to 17.05%, illustrating the influence of prior hospital utilization without establishing causation.

Clinical Interpretation and Limitations

The models demonstrated modest predictive ability, suggesting that routinely collected clinical information contains some useful information about readmission risk. However, the relatively low sensitivity and positive predictive value limit their clinical usefulness. The dataset is historical, contains substantial missing information in several variables, and may not capture readmissions outside participating hospitals. The results should therefore be interpreted as an exploratory prediction study.

Conclusion

Machine learning models can identify some patterns associated with 30-day hospital readmission among patients with diabetes, but the predictive performance achieved in this project is insufficient to support independent clinical decision-making. Further validation, improved clinical data, and additional research would be necessary before considering clinical implementation.

Double-click (or enter) to edit